Diraschool — REST API Reference

Diraschool — REST API Reference

> **Version:** 1.0
> **Production URL:** `https://api.diraschool.co.ke/api/v1`
> **Local dev:** `http://localhost:5000/api/v1`

Table of Contents

1. Overview
2. Deployment & Hosting
3. Security
4. Authentication & Cookies
5. mustChangePassword Flow
6. Standard Response Format
7. Pagination
8. Error Codes
9. Rate Limiting
10. Subscription Plans & Feature Gates
11. Roles & Permissions
12. Health
13. Auth
14. Users
15. Schools
16. Classes
17. Students
18. Attendance
19. Subjects
20. Exams
21. Results
22. Fees
23. Report Cards *(plan-gated)*
24. Parent Portal *(plan-gated)*
25. Audit Logs *(plan-gated)*
26. School Settings
27. Timetables *(plan-gated)*
28. Library *(plan-gated)*
29. Transport *(plan-gated)*

Overview

Diraschool is a **multi-tenant** SaaS platform for Kenyan CBC schools.
Every school is a separate tenant — all data is scoped by `schoolId`.
Superadmin users can manage all tenants from a single account.

All request and response bodies are **JSON** (`Content-Type: application/json`).
File uploads use **multipart/form-data**.

Deployment & Hosting

Railway.app *(recommended)*

- Subdomain: `*.railway.app`
- No spin-down — the API stays warm 24/7 (important for BullMQ background workers)
- GitHub auto-deploy on push to main
- $5/month hobby plan
- Required add-ons: **MongoDB Atlas** (free M0 tier) + **Upstash Redis** (free tier)

Render.com *(good alternative)*

- Subdomain: `*.onrender.com`
- Free tier spins down after 15 min of inactivity — **do not use the free tier** for an API with BullMQ workers; use the $7/month paid plan
- GitHub auto-deploy supported

Custom Domain

Add a custom domain (`api.diraschool.co.ke`) in the Railway/Render dashboard at no extra cost, then point your DNS A/CNAME records once the domain is acquired.

Required Environment Variables

Set these on the hosting platform:

Variable	Notes
`NODE_ENV`	`production`
`MONGO_URI`	Atlas connection string
`REDIS_URL`	Upstash Redis URL
`JWT_SECRET`	Minimum 32 random characters (`openssl rand -hex 32`)
`CLIENT_URL`	Frontend origin (used for CORS)
`PORT`	Railway/Render auto-set this — do not hardcode

Security

All requests pass through the following security layers in order:

Layer	Implementation	Purpose
Helmet	`helmet()` middleware	Sets secure HTTP headers (CSP, HSTS, etc.)
CORS whitelist	`cors({ origin: CLIENT_URL })`	Only allows requests from known frontend origins
JWT cookie	HTTP-only, Secure, SameSite=Strict	Prevents XSS token theft and CSRF via cross-site form posts
Origin/Referer CSRF middleware	Custom middleware checks `Origin`/`Referer` header	Rejects requests that don't originate from the whitelisted frontend
Per-school rate limiting	Redis-backed; 300 req/60 s per school	Prevents API abuse and limits blast radius of a single school
Auth rate limiting	20 req/15 min on login/register	Limits brute-force attacks

---

Authentication & Cookies

Authentication uses **HTTP-only cookies** (no `Authorization` header needed from browsers).
The cookie name is `token` and it contains a signed JWT.

Cookie	Type	TTL
`token`	HTTP-only, Secure, SameSite=Strict	1 day

Include credentials on every request:

```
fetch('/api/v1/...', { credentials: 'include' })
```

For API clients (Postman, mobile), the cookie is set automatically on login.

---

mustChangePassword Flow

When a staff user is created by an admin (via `POST /users`), or when a parent account is auto-created during student enrollment, the flag `mustChangePassword: true` is set on the user.

- The **login response** includes `"mustChangePassword": true` when this flag is set.

- Until the user calls `POST /auth/change-password`, **all other endpoints return `HTTP 403`** with the message `"Password change required before continuing"`.
- `POST /auth/change-password` and `POST /auth/logout` are the only endpoints that remain accessible.

Clients should detect `mustChangePassword: true` on login and immediately redirect to a change-password screen.

Standard Response Format

Success:

```
{
  "success": true,
  "data": { ... }
}
```

Success with pagination:

```
{
  "success": true,
  "data": {
    "items": [...],
    "meta": {
      "page": 1,
      "limit": 20,
      "total": 145,
      "pages": 8
    }
  }
}
```

Error:

```
{
  "success": false,
  "message": "Human-readable error description"
}
```

Pagination

List endpoints accept these query parameters:

Param	Type	Default	Description
`page`	integer	1	Page number (1-based)
`limit`	integer	20	Items per page (max 100)

Response always includes a `meta` object:

```
"meta": { "page": 1, "limit": 20, "total": 145, "pages": 8 }
```

Error Codes

Status	Meaning
400	Validation error — check `message` for details
401	Not authenticated — login required
403	Forbidden — wrong role, plan gate, account suspended, or password change required
404	Resource not found
409	Conflict — duplicate record or business rule violation
422	Unprocessable — logically invalid request
429	Rate limited — slow down
500	Internal server error

Rate Limiting

Scope	Limit	Window
Per-school (all users combined)	300 requests	60 seconds
Auth endpoints (login/register)	20 requests	15 minutes

Hitting the limit returns `HTTP 429` with `{ "retryAfter": 60 }`.

Subscription Plans & Feature Gates

Schools have a `planTier` (trial / basic / standard / premium) and a `subscriptionStatus` (trial / active / suspended / expired).

Subscription status determines if the school can access the API at all.
Plan tier determines which feature modules are available.

> Pricing and exact tier allocations are **TBD**. All features are currently available on all tiers (including trial) — feature gates are wired but open. Once pricing is set, update `PLAN_FEATURE_MAP` in `src/constants/index.js`.

Feature Module	Feature Key	Gated Endpoints
Report Cards	`report_cards`	`/report-cards/*`
Parent Portal	`parent_portal`	`/parent/*`
Timetable	`timetable`	`/timetables/*`

Feature Module	Feature Key	Gated Endpoints
Library	`library`	`/library/*`
Transport	`transport`	`/transport/*`
Bulk Import	`bulk_import`	`POST /students/import`
Audit Log	`audit_log`	`/audit-logs/*`
SMS	`sms`	`/sms/*` *(not yet built)*

A blocked feature request returns:

```
HTTP 403
{
  "message": "The \"library\" feature is not available on your current
        plan (basic). Please upgrade.",
  "feature": "library",
  "currentPlan": "basic",
  "upgradeRequired": true
}
```

Roles & Permissions

Role	Scope	Description
`superadmin`	Platform	Full access to all schools and superadmin endpoints
`school_admin`	School	Full access within school
`director`	School	Full access within school
`headteacher`	School	Full access within school
`deputy_headteacher`	School	Full access within school
`secretary`	School	Read/write most data, no financial
`accountant`	School	Fees and payments only
`teacher`	School	Own subjects, results, attendance
`parent`	School	Read-only: own children's data

Admin roles = `school_admin`, `director`, `headteacher`, `deputy_headteacher`

Health

`GET /health`

Service liveness check. No authentication required. Used by hosting platforms (Railway/Render) and uptime monitors.

Auth: None (public)

Response: `200`

```
{
  "status": "ok",
  "timestamp": "2026-04-10T08:00:00.000Z"
}
```

Auth

`POST /auth/register`

Create a new school and its first admin user. Password must be **at least 8 characters**.

Auth: None (public)

Body:

```
{
  "schoolName": "Nairobi Primary School",
  "schoolEmail": "admin@nairobiprimary.co.ke",
  "schoolPhone": "+254712345678",
  "county": "Nairobi",
  "adminFirstName": "Jane",
  "adminLastName": "Doe",
  "adminEmail": "jane.doe@nairobiprimary.co.ke",
  "adminPassword": "SecurePass123!"
}
```

Response: `201` — Sets `token` cookie (1-day TTL). Returns `{ user, school }`.

`POST /auth/login`

Log in and receive a session cookie.

Auth: None (public)

Body:

```
{
  "email": "jane.doe@nairobiprimary.co.ke",
  "password": "SecurePass123!"
}
```

Response: `200` — Sets `token` cookie. Returns `{ user }`.

> If the user has `mustChangePassword: true`, the response body includes `"mustChangePassword": true`. See mustChangePassword Flow.

`POST /auth/logout`

Clear the session cookie.

Auth: Cookie (any authenticated user)

Response: `200` — Clears `token` cookie.

`GET /auth/me`

Returns the currently authenticated user.

Auth: Cookie

Response: `200` — Returns `{ user }`.

`POST /auth/change-password`

Change your own password. Clears the `mustChangePassword` flag. New password must be **at least 8 characters**.

Auth: Cookie (accessible even when `mustChangePassword: true`)

Body:

```
{
  "currentPassword": "OldPassword1",
  "newPassword": "NewPassword1"
}
```

Response: `200`

Users

> Admin roles only (except `GET /users/:id` which any school user can call for their own profile).

`POST /users`

Create a school staff user (not a parent — parents are created via student enrollment). Sets `mustChangePassword: true` on the created user. Password must be **at least 8 characters**.

Auth: Admin

Body:

```
{
  "firstName": "John",
  "lastName": "Kamau",
  "email": "j.kamau@school.co.ke",
  "phone": "+254722000001",
  "role": "teacher",
  "password": "TempPass123"
}
```

Response: `201` — Returns `{ user }`.

`GET /users`

List all users in the school.

Auth: Admin

Query: `?role=teacher&isActive=true&page=1&limit=20`

Response: `200` — Returns `{ users, meta }`.

`GET /users/:id`

Get a single user.

Auth: Any school user

Response: `200` — Returns `{ user }`.

`PATCH /users/:id`

Update a user's details or activate/deactivate them.

Auth: Admin

Body (all fields optional):

```
{
  "firstName": "John",
  "lastName": "Kamau",
  "phone": "+254722000001",
  "role": "headteacher",
  "isActive": false
}
```

Response: `200` — Returns `{ user }`.

Schools

The school object includes a `planTier` field (`trial` | `basic` | `standard` | `premium`) alongside `subscriptionStatus`.

`GET /schools/me`

Get the logged-in user's school profile.

Auth: Any school user

Response: `200` — Returns `{ school }`.

`PATCH /schools/me`

Update the school's own profile (non-sensitive fields only).

Auth: Admin

Body (all optional):

```
{
  "name": "Updated School Name",
  "phone": "+254700000001",
  "county": "Kiambu",
  "registrationNumber": "KE/2024/001",
  "address": "P.O Box 123, Thika"
}
```

Response: `200` — Returns `{ school }`.

`POST /schools` *(superadmin)*

Create a new school tenant.

Auth: Superadmin

Body:

```
{
  "name": "Mombasa Academy",
  "email": "admin@mombasaacademy.co.ke",
  "phone": "+254711000002",
  "county": "Mombasa"
}
```

Response: `201` — Returns `{ school }`.

`GET /schools` *(superadmin)*

List all school tenants.

Auth: Superadmin

Query: `?active=true&subscriptionStatus=trial&page=1&limit=20`

Response: `200` — Returns `{ schools, meta }`.

`GET /schools/:id` *(superadmin)*

Get a single school.

Auth: Superadmin

Response: `200` — Returns `{ school }`.

`PATCH /schools/:id` *(superadmin)*

Update school details including `isActive` toggle.

Auth: Superadmin

Body (all optional):

```
{
  "name": "New Name",
  "email": "new@school.co.ke",
  "isActive": false
}
```

Response: `200` — Returns `{ school }`.

`PATCH /schools/:id/subscription` *(superadmin)*

Change a school's subscription status and/or plan tier.

Auth: Superadmin

Body:

```
{
  "subscriptionStatus": "active",
  "planTier": "standard",
  "trialExpiry": "2025-03-31"
}
```

`subscriptionStatus` values: `trial` | `active` | `suspended` | `expired`
`planTier` values: `trial` | `basic` | `standard` | `premium`

Response: `200` — Returns `{ school, message }`.

Classes

`POST /classes`

Create a new class.

Auth: Admin

Body:

```
{
  "name": "Grade 4",
  "stream": "Blue",
  "levelCategory": "Upper Primary",
  "academicYear": "2025",
  "term": "Term 1"
}
```

`levelCategory` values: `Pre-Primary` | `Lower Primary` | `Upper Primary` | `Junior Secondary` | `Senior School`

Response: `201` — Returns `{ class }`.

`GET /classes`

List classes (Redis-cached for 10 min on first page, unfiltered queries).

Auth: Any school user

Query: `?levelCategory=Upper+Primary&academicYear=2025&page=1&limit=20`

Response: `200` — Returns `{ classes, meta }`.

`GET /classes/:id`

Get a single class.

Auth: Any school user

Response: `200` — Returns `{ class }`.

`PATCH /classes/:id`

Update class details.

Auth: Admin

Body (all optional): Same fields as POST.

Response: `200` — Returns `{ class }`.

`DELETE /classes/:id`

Delete a class. Blocked if students are enrolled.

Auth: Admin

Response: `200`

`POST /classes/:id/promote`

Bulk promote all active students in this class to a target class.

Auth: Admin

Body:

```
{
  "targetClassId": "64f..."
}
```

Response: `200` — Returns `{ modifiedCount, message }`.

Students

Student objects include a `routeId` field (ObjectId, optional) — this is populated when a transport route has been assigned to the student via `POST /transport/routes/:id/assign`.

`POST /students`

Enroll a new student. Optionally create or link a parent user.
If a new parent user is created, `mustChangePassword: true` is set on the parent account.

Auth: Admin

Body:

```
{
  "classId": "64f...",
  "admissionNumber": "ADM2025001",
  "firstName": "Alice",
```

```
  "lastName": "Wanjiku",
  "gender": "female",
  "dateOfBirth": "2014-03-15",
  "parent": {
    "firstName": "Mary",
    "lastName": "Wanjiku",
    "phone": "+254712345678",
    "email": "mary.wanjiku@gmail.com"
  }
}
```

To link an existing parent user instead of creating one:

```
"parent": { "existingUserId": "64f..." }
```

Response: `201` — Returns `{ student }` with parent populated.

`GET /students`

List students.

Auth: Admin

Query: `?classId=64f...&status=active&search=Alice&page=1&limit=20`

`status` values: `active` | `transferred` | `graduated` | `withdrawn`

Response: `200` — Returns `{ students, meta }`.

`GET /students/:id`

Get a student with class and parent details.

Auth: Admin

Response: `200` — Returns `{ student }`.

`PATCH /students/:id`

Update basic student details.

Auth: Admin

Body (all optional): `firstName`, `lastName`, `gender`, `dateOfBirth`, `admissionNumber`

Response: `200` — Returns `{ student }`.

`POST /students/:id/transfer`

Move a student to a different class within the same school.

Auth: Admin

Body:

```
{
  "newClassId": "64f...",
  "note": "Transferred due to stream reshuffle"
}
```

Response: `200`

`POST /students/:id/withdraw`

Soft-delete a student (marks as withdrawn, decrements class count).

Auth: Admin

Response: `200`

`POST /students/import` *(plan-gated: bulk_import)*

Upload a CSV file to bulk-import students. Returns a `jobId` immediately; poll status endpoint for results.

Auth: Admin
Content-Type: `multipart/form-data`

Form fields: - `file` — CSV file (max 5 MB) - `classId` — target class ObjectId

CSV format (header row required):

```
admissionNumber,firstName,lastName,gender,dateOfBirth,parentFirstName,
ADM001,Alice,Wanjiku,female,2014-03-
15,Mary,Wanjiku,+254712000001,mary@gmail.com
```

Response: `202`

```
{
  "jobId": "42",
  "total": 150,
  "preValidationErrors": []
}
```

`GET /students/import/:jobId/status`

Poll the result of a bulk import job.

Auth: Admin

Response: `200` when done, `202` while still processing.

```
{
  "result": {
    "total": 150,
    "created": 148,
    "failed": 2,
    "errors": [
      { "row": 14, "error": "Duplicate admission number ADM013" }
    ]
  }
}
```

Attendance

`POST /attendance`

Create or update the attendance register for a class on a date.

Auth: Admin / Teacher

Body:

```
{
  "classId": "64f...",
  "date": "2025-03-10",
  "term": "Term 1",
  "academicYear": "2025",
  "entries": [
    { "studentId": "64f...", "status": "present" },
    { "studentId": "64f...", "status": "absent", "note": "Sick" }
  ]
}
```

`status` values: `present` | `absent` | `late` | `excused`

Response: `201` or `200` (upsert).

`GET /attendance`

List attendance registers.

Auth: Admin / Teacher

Query: `?classId=64f...&date=2025-03-10&term=Term+1&academicYear=2025&status=submitted`

Response: `200` — Returns `{ registers, meta }`.

`GET /attendance/:id`

Get a single register with all entries.

Auth: Admin / Teacher

Response: `200` — Returns `{ register }`.

`PATCH /attendance/:id`

Update entries on a draft register.

Auth: Admin / Teacher

Body:

```
{
  "entries": [
    { "studentId": "64f...", "status": "late" }
  ]
}
```

Response: `200`

`POST /attendance/:id/submit`

Submit (lock) a register. Blocked if already submitted.

Auth: Admin / Teacher

Response: `200`

Subjects

`POST /subjects`

Add a subject to a class.

Auth: Admin

Body:

```
{
  "classId": "64f...",
  "name": "Mathematics",
  "code": "MTH",
  "teacherId": "64f..."
}
```

Response: `201` — Returns `{ subject }`.

`GET /subjects`

List subjects.

Auth: Any school user

Query: `?classId=64f...`

Response: `200` — Returns `{ subjects }`.

`GET /subjects/:id`

Get a single subject.

Auth: Any school user

Response: `200` — Returns `{ subject }`.

`PATCH /subjects/:id`

Update a subject (name, code, teacherId).

Auth: Admin

Body (all optional): `name`, `code`, `teacherId`

Response: `200`

`DELETE /subjects/:id`

Delete a subject. Blocked if exam results exist.

Auth: Admin

Response: `200`

Exams

`POST /exams`

Create an exam for a class + subject.

Auth: Admin

Body:

```
{
  "classId": "64f...",
  "subjectId": "64f...",
  "name": "Term 1 Opener",
  "type": "opener",
  "totalMarks": 100,
  "academicYear": "2025",
  "term": "Term 1",
  "date": "2025-02-10"
}
```

`type` values: `opener` | `midterm` | `endterm` | `sba`

Response: `201` — Returns `{ exam }`.

`GET /exams`

List exams.

Auth: Any school user

Query: ?
`classId=64f...&subjectId=64f...&term=Term+1&academicYear=2025&type=endterm`

Response: `200` — Returns `{ exams, meta }`.

`GET /exams/:id`

Get a single exam.

Auth: Any school user

Response: `200`

`PATCH /exams/:id`

Update exam metadata. `totalMarks` change blocked if results exist.

Auth: Admin

Body (all optional): `name`, `type`, `totalMarks`, `date`

Response: `200`

`DELETE /exams/:id`

Delete an exam. Blocked if results have been entered.

Auth: Admin

Response: `200`

Results

`POST /results/bulk`

Enter or update marks for multiple students in one exam.

Auth: Admin / Teacher

Body:

```
{
  "examId": "64f...",
  "entries": [
    { "studentId": "64f...", "marks": 78 },
    { "studentId": "64f...", "marks": 55 }
  ]
}
```

Grades are computed automatically using the CBC rubric for the class's `levelCategory`: - **Lower/Upper Primary (Gr 1–6):** EE / ME / AE / BE - **Junior/Senior Secondary (Gr 7–12):** EE1–BE2 (8-point KNEC scale)

Response: `201` — Returns `{ results }`.

`GET /results`

List results.

Auth: Any school user

Query: ?
`examId=64f...&studentId=64f...&subjectId=64f...&term=Term+1&academicYear=2025`

Response: `200` — Returns `{ results, meta }`.

`GET /results/:id`

Get a single result.

Auth: Any school user

Response: `200`

`PATCH /results/:id`

Update marks for a single result (recalculates grade).

Auth: Admin / Teacher

Body:

```
{ "marks": 82 }
```

Response: `200`

Fees

`POST /fees/structures`

Define the fee schedule for a class + term.

Auth: Admin / Accountant

Body:

```
{
  "classId": "64f...",
  "academicYear": "2025",
  "term": "Term 1",
  "items": [
    { "name": "Tuition", "amount": 15000 },
    { "name": "Activity Fee", "amount": 2500 }
  ]
}
```

Response: `201` — `totalAmount` is auto-calculated.

`GET /fees/structures`

List fee structures.

Auth: Admin / Accountant

Query: `?classId=64f...&academicYear=2025&term=Term+1`

Response: `200` — Returns `{ structures, meta }`.

`GET /fees/structures/:id`

Auth: Admin / Accountant

Response: `200`

`PATCH /fees/structures/:id`

Replace fee items (recalculates total).

Auth: Admin / Accountant

Body:

```
{
  "items": [
    { "name": "Tuition", "amount": 16000 }
  ]
}
```

Response: `200`

`DELETE /fees/structures/:id`

Delete a fee structure. Blocked if payments exist for the same class/term/year.

Auth: Admin / Accountant

Response: `200`

`POST /fees/payments`

Record a fee payment for a student. Automatically enqueues a PDF receipt job.

Auth: Admin / Accountant

Body:

```
{
  "studentId": "64f...",
  "academicYear": "2025",
  "term": "Term 1",
  "amount": 10000,
  "method": "mpesa",
  "reference": "QHE2XXXX",
  "notes": "Partial payment"
}
```

`method` values: `cash` | `mpesa` | `bank`

Response: `201` — Returns `{ payment }`.

> **`receiptUrl` note:** The `receiptUrl` field on the payment is initially `null`. It is populated asynchronously (typically within a few seconds) once the PDF receipt worker generates the receipt and uploads it to Cloudinary. If you need the PDF link immediately after recording a payment, poll `GET /fees/payments/:id` until `receiptUrl` is non-null.

`GET /fees/payments`

List payments.

Auth: Admin / Accountant

Query: `?studentId=64f...&classId=64f...&academicYear=2025&term=Term+1&method=mpesa&status=completed`

Response: `200` — Returns `{ payments, meta }`.

`GET /fees/payments/:id`

Auth: Admin / Accountant

Response: `200`

`POST /fees/payments/:id/reverse`

Void a completed payment (marks as reversed, does not delete).

Auth: Admin / Accountant

Body:

```
{
  "reversalReason": "Duplicate payment recorded in error"
}
```

Response: `200`

`GET /fees/balance`

Get a student's outstanding fee balance for a term.

Auth: Admin / Accountant

Query: `?studentId=64f...&academicYear=2025&term=Term+1`

Response:

```
{
  "totalPaid": 10000,
  "outstanding": 7500,
  "overpaid": 0,
  "isPaidUp": false,
  "feeStructure": { "totalAmount": 17500, "items": [...] }
}
```

Report Cards

> **Plan-gated:** `report_cards`

`POST /report-cards/generate`

Generate (or regenerate) a report card for a single student + term.

Auth: Admin

Body:

```
{
  "studentId": "64f...",
  "academicYear": "2025",
  "term": "Term 1"
}
```

Response: `201` or `200` (upsert). Returns `{ reportCard }`.
A published report card **cannot** be regenerated.

`POST /report-cards/generate-class`

Bulk-generate report cards for all active students in a class.

Auth: Admin

Body:

```
{
  "classId": "64f...",
  "academicYear": "2025",
  "term": "Term 1"
}
```

Response: `200` — Returns `{ generated, skipped, errors }`.

`GET /report-cards`

List report cards.

Auth: Admin

Query: `?classId=64f...&studentId=64f...&academicYear=2025&term=Term+1&status=published`

Response: `200` — Returns `{ reportCards, meta }`.

`GET /report-cards/annual-summary`

Cross-term comparison for one student across all three terms.

Auth: Admin

Query (both params are required):

Param	Type	Required	Description
`studentId`	ObjectId	**Yes**	The student to summarise
`academicYear`	string	**Yes**	e.g. `2025`

Response: Returns all available term report cards plus a per-subject `annualAvgPct`.

`GET /report-cards/:id`

Get a single report card.

Auth: Admin

Response: `200` — Full report card with subject summaries and attendance.

`POST /report-cards/:id/publish`

Lock a report card. Blocked if no subject results. After publishing, PDF generation is enqueued.

Auth: Admin

Response: `200`

`PATCH /report-cards/:id/remarks`

Update the overall teacher and principal remarks.

Auth: Admin
Blocked on published cards.

Body:

```
{
  "teacherRemarks": "Excellent effort this term.",
  "principalRemarks": "Keep it up!"
}
```

Response: `200`

`PATCH /report-cards/:id/subjects/:subjectId/remark`

Add or clear a per-subject teacher remark.

Auth: Admin
Blocked on published cards.

Body:

```
{ "remark": "Shows strong analytical skills." }
```

Pass an empty string or `null` to clear the remark.

Response: `200`

Parent Portal

> **Plan-gated:** `parent_portal`
> All endpoints require role `parent`.

`GET /parent/children`

List all students linked to the logged-in parent.

Auth: Parent

Response: `200` — Returns `{ children }`.

`GET /parent/children/:studentId/fees`

Get fee payments and balance for a child.

Auth: Parent (must be linked to the student)

Query: `?academicYear=2025&term=Term+1`

Response: `200` — Returns `{ payments, balance }`.

`GET /parent/children/:studentId/attendance`

Get attendance records for a child.

Auth: Parent

Query: `?term=Term+1&academicYear=2025`

Response: `200` — Returns `{ registers }`.

`GET /parent/children/:studentId/results`

Get exam results for a child.

Auth: Parent

Query: `?academicYear=2025&term=Term+1`

Response: `200` — Returns `{ results }`.

`GET /parent/children/:studentId/report-cards`

Get published report cards for a child (drafts are hidden from parents).

Auth: Parent

Query: `?academicYear=2025`

Response: `200` — Returns `{ reportCards }`.

Audit Logs

> **Plan-gated:** `audit_log`
> Admin-only read access.

`GET /audit-logs`

List audit log entries for the school, newest first.

Auth: Admin

Query:

Param	Description
`resource`	Filter by resource type (e.g. `Payment`, `Student`)
`action`	Filter by action (e.g. `create`, `reverse`, `publish`)
`userId`	Filter by the user who performed the action
`resourceId`	Filter by a specific document ID
`from`	ISO date — earliest `createdAt`
`to`	ISO date — latest `createdAt`
`page`, `limit`	Pagination

Response: `200` — Returns `{ logs, meta }`.

Logged actions:

Trigger	Action	Resource
Enroll student	`create`	`Student`
Transfer student	`transfer`	`Student`
Withdraw student	`withdraw`	`Student`
Record payment	`create`	`Payment`
Reverse payment	`reverse`	`Payment`
Publish report card	`publish`	`ReportCard`
Toggle school isActive	`activate/suspend`	`School`
Update subscription	`update`	`School`
Issue book	`issue`	`BookLoan`
Return book	`return`	`BookLoan`

School Settings

`GET /settings`

Get school settings. Creates a default document on first call (upsert).
Result is Redis-cached for 30 minutes.

Auth: Admin

Response:

```
{
  "settings": {
    "currentAcademicYear": "2025",
    "terms": [
      { "name": "Term 1", "startDate": "2025-01-06", "endDate": "2025-
        04-04" },
      { "name": "Term 2", "startDate": "2025-05-05", "endDate": "2025-
        08-01" },
      { "name": "Term 3", "startDate": "2025-09-01", "endDate": "2025-
        11-14" }
    ],
    "workingDays":
        ["monday","tuesday","wednesday","thursday","friday"],
    "holidays": [],
    "motto": "Excellence in All We Do",
    "principalName": "Dr. Jane Otieno",
    "physicalAddress": "P.O Box 100, Nairobi"
  }
}
```

`PUT /settings`

Update settings fields (partial update — only provided fields are changed).

Auth: Admin

Body (all optional):

```
{
  "currentAcademicYear": "2025",
  "terms": [
    { "name": "Term 1", "startDate": "2025-01-06", "endDate": "2025-
        04-04" }
  ],
  "workingDays": ["monday","tuesday","wednesday","thursday","friday"],
  "motto": "Excellence in All We Do",
  "principalName": "Dr. Jane Otieno",
  "physicalAddress": "P.O Box 100, Nairobi"
}
```

Response: `200` — Returns `{ settings }`.

`POST /settings/holidays`

Add a holiday to the school calendar.

Auth: Admin

Body:

```
{
  "name": "Madaraka Day",
  "date": "2025-06-01",
  "description": "National holiday"
}
```

Response: `201` — Returns `{ holiday, settings }`.

`DELETE /settings/holidays/:holidayId`

Remove a holiday by its subdocument `_id`.

Auth: Admin

Response: `200`

Timetables

> **Plan-gated:** `timetable`

`POST /timetables`

Create a timetable for a class + term + year. One timetable per class per term per year (unique constraint).

Auth: Admin

Body:

```
{
  "classId": "64f...",
  "academicYear": "2025",
  "term": "Term 1",
  "slots": [
    {
      "day": "monday",
      "period": 1,
      "startTime": "08:00",
      "endTime": "08:40",
      "subjectId": "64f...",
      "teacherId": "64f...",
      "room": "Lab 2"
    }
  ]
}
```

Response: `201` — Returns `{ timetable }` with subjects and teachers populated.

`GET /timetables`

List timetables.

Auth: Admin, Deputy, Headteacher, Teacher, Secretary

Query: `?classId=64f...&academicYear=2025&term=Term+1`

Response: `200` — Returns `{ timetables, meta }`.

`GET /timetables/:id`

Get a full timetable with all slot details populated.

Auth: Admin, Deputy, Headteacher, Teacher, Secretary

Response: `200` — Returns `{ timetable }`.

`PUT /timetables/:id/slots`

Replace all slots in a timetable (full overwrite).

Auth: Admin

Body:

```
{
  "slots": [
    {
      "day": "tuesday",
      "period": 2,
      "startTime": "09:00",
      "endTime": "09:40",
      "subjectId": "64f...",
      "teacherId": "64f...",
      "room": "Room 3"
    }
  ]
}
```

Response: `200` — Returns `{ timetable }`.

`DELETE /timetables/:id`

Delete a timetable.

Auth: Admin

Response: `200`

Library

> **Plan-gated:** `library`

`POST /library/books`

Add a book to the catalogue.

Auth: Admin

Body:

```
{
  "title": "Mathematics Grade 7",
  "author": "KLB Publishers",
  "isbn": "978-9966-47-123-4",
  "category": "Textbook",
  "totalCopies": 30
}
```

`availableCopies` is automatically set to `totalCopies` on creation.

Response: `201` — Returns `{ book }`.

`GET /library/books`

List books.

Auth: Any school staff

Query: ?
`category=Textbook&isActive=true&search=mathematics&page=1&limit=20`

Response: `200` — Returns `{ books, meta }`.

`GET /library/books/:id`

Get a single book.

Auth: Any school staff

Response: `200`

`PATCH /library/books/:id`

Update book details. Changing `totalCopies` proportionally adjusts `availableCopies`.

Auth: Admin

Body (all optional): `title`, `author`, `isbn`, `category`, `totalCopies`, `isActive`

Response: `200`

`POST /library/loans`

Issue a book to a student or staff member.
Uses a MongoDB transaction to atomically decrement `availableCopies`.
Returns `409` if no copies are available.

Auth: Admin, Teacher, Secretary

Body:

```
{
  "bookId": "64f...",
  "borrowerType": "student",
  "borrowerId": "64f...",
  "borrowerName": "Alice Wanjiku",
  "dueDate": "2025-04-15",
  "notes": "For holiday reading"
}
```

`borrowerType` values: `student` | `staff`

Response: `201` — Returns `{ loan }`.

`GET /library/loans`

List loans.

Auth: Any school staff

Query: ?
`bookId=64f...&borrowerId=64f...&borrowerType=student&status=active&page=1&limit=20`

`status` values: `active` | `returned` | `overdue`

Response: `200` — Returns `{ loans, meta }`.

`GET /library/loans/:id`

Get a single loan.

Auth: Any school staff

Response: `200`

`POST /library/loans/:id/return`

Mark a loan as returned. Atomically increments `availableCopies`.

Auth: Admin, Teacher, Secretary

Body (optional):

```
{ "notes": "Returned in good condition" }
```

Response: `200` — Returns `{ loan }`.

`PATCH /library/loans/:id/overdue`

Manually mark an active loan as overdue.

Auth: Admin

Response: `200`

> **Tip:** In production, run a nightly scheduled job (`CronJob`) that marks all loans where `dueDate < now` and `status = active` as `overdue`.

Transport

> **Plan-gated:** `transport`

`POST /transport/routes`

Create a new transport route.

Auth: Admin

Body:

```
{
  "name": "Route A — Westlands",
  "description": "Morning and afternoon route via Westlands",
  "vehicleReg": "KBZ 123A",
  "driverName": "Peter Mwangi",
  "driverPhone": "+254722000001",
  "capacity": 40,
  "morningDeparture": "06:30",
  "afternoonDeparture": "17:00",
  "stops": [
    { "name": "Westlands Stage", "order": 1, "lat": -1.2676, "lng":
        36.8027 },
```

```
    { "name": "Parklands", "order": 2, "lat": -1.2572, "lng": 36.8141
        }
  ]
}
```

Response: `201` — Returns `{ route }`.

`GET /transport/routes`

List routes.

Auth: Any school staff

Query: `?isActive=true&page=1&limit=20`

Response: `200` — Returns `{ routes, meta }`.

`GET /transport/routes/:id`

Get a route plus a list of students currently assigned to it.

Auth: Any school staff

Response:

```
{
  "route": { ... },
  "students": [
    {
      "_id": "64f...",
      "firstName": "Alice",
      "lastName": "Wanjiku",
      "admissionNumber": "ADM001",
      "classId": { "name": "Grade 4", "stream": "Blue" }
    }
  ]
}
```

`PATCH /transport/routes/:id`

Update route details.

Auth: Admin

Body (all optional): Any field from the create body.

Response: `200`

`DELETE /transport/routes/:id`

Delete a route. Blocked if any students are currently assigned to it.

Auth: Admin

Response: `200`

`POST /transport/routes/:id/assign`

Bulk-assign students to this route (sets `Student.routeId`).

Auth: Admin

Body:

```
{
  "studentIds": ["64f...", "64f...", "64f..."]
}
```

Response: `200` — Returns `{ modifiedCount, message }`.

`POST /transport/routes/:id/unassign`

Remove students from this route (unsets `Student.routeId`).

Auth: Admin

Body:

```
{
  "studentIds": ["64f...", "64f..."]
}
```

Response: `200` — Returns `{ modifiedCount, message }`.

CBC Grading Reference

Lower Primary & Upper Primary (Grade 1–6)

Grade	Label	Description
EE	Exceeds Expectations	Outstanding performance
ME	Meets Expectations	Satisfactory performance
AE	Approaching Expectations	Needs improvement
BE	Below Expectations	Significantly below level

Junior Secondary & Senior School (Grade 7–12)

Points	Grade	Percentage Range
8	EE1	90–100%
7	EE2	75–89%
6	ME1	58–74%
5	ME2	41–57%
4	AE1	31–40%
3	AE2	21–30%
2	BE1	11–20%
1	BE2	1–10%

Constants Reference

Terms

`"Term 1" | "Term 2" | "Term 3"`

Payment Methods

`cash | mpesa | bank`

Subscription Statuses

`trial | active | suspended | expired`

Plan Tiers

`trial | basic | standard | premium`

Days of Week

`monday | tuesday | wednesday | thursday | friday | saturday | sunday`

Student Statuses

`active | transferred | graduated | withdrawn`

Loan Statuses

`active | returned | overdue`

Borrower Types

`student` | `staff`

Exam Types

`opener` | `midterm` | `endterm` | `sba`

Attendance Statuses

`present` | `absent` | `late` | `excused`

Last updated: April 2026